

Technical Design Document (TDD)

TapConnect

Version: 1.0

Date: 2026-05-14

Platform: Android

Primary Purpose: Define the system architecture, module design, data model, technical flows, and implementation strategy for TapConnect.

1. Executive Summary

TapConnect is an Android-based proximity networking application that enables users to connect instantly using **NFC tap-to-connect** and **BLE nearby discovery**. The system is designed to be lightweight, privacy-first, and event-friendly, with optional AI-powered enhancements for smarter networking insights.

This document describes how the application will be structured, how modules communicate, how user data is managed, and how the product will scale from a local MVP into a backend-supported platform.

2. Design Goals

The technical design is intended to achieve the following goals:

- Support fast proximity-based connections.
 - Keep the first version Android-only.
 - Make the app usable with minimal interaction during live events.
 - Ensure privacy through consent-based discoverability.
 - Keep the architecture modular so NFC, BLE, AI, and backend features can evolve independently.
 - Support a local-first MVP while allowing future cloud expansion.
-

3. Assumptions

This design is based on the following assumptions:

- The first release targets Android only.
- The app must be open and active during networking.
- Users will manually enable Networking Mode.
- User profiles are stored locally for the MVP.
- NFC and BLE support will vary by device, so fallback paths are required.
- AI features in the MVP will be rule-based or lightweight.

- A backend may be added later for sync, analytics, and shared profiles.
-

4. System Overview

TapConnect follows a **client-centric architecture** where each Android device acts as both a broadcaster and scanner depending on the connection method.

Core Design Principles

- **Local-first:** Profile data and preferences remain on-device initially.
 - **Intent-based sharing:** No automatic sharing without user action.
 - **Modular connectivity:** NFC and BLE are separate connectivity modules.
 - **Minimal latency:** Discovery and connection flows are optimized for event usage.
 - **Extensible AI layer:** AI logic starts simple and can later be replaced by API-driven intelligence.
-

5. High-Level Architecture

Main Layers

1. UI Layer

Screens, user interaction, animations, status indicators.

2. Application Layer

ViewModels, state management, connection orchestration, validation.

3. Domain Layer

Business rules for matching, visibility, connection handling, privacy constraints.

4. Data Layer

Local storage, profile persistence, connection history, optional remote sync.

5. Connectivity Layer

NFC manager, BLE scanner/broadcaster, permissions, session exchange.

6. AI Layer

Summary generation, icebreakers, interest overlap detection, relevance ranking.

6. Core Modules

6.1 Profile Module

Responsibility

Manages user identity, profile content, and sharing preferences.

Functions

- Create user profile
- Update profile information
- Store user interests and links
- Manage visibility preferences
- Return shared profile data based on privacy rules

Data Managed

- user_id
- name
- profile_image
- bio
- role/title
- organization
- interests
- social_links
- sharing_preferences

Notes

- The module should separate private profile fields from public sharing fields.
 - Sharing preferences must be independently editable.
-

6.2 BLE Module

Responsibility

Handles nearby discovery, broadcasting, scanning, and BLE-based fallback connectivity.

Functions

- Start BLE scan when Networking Mode is active
- Broadcast discoverable presence
- Detect nearby active users
- Stop scanning when Networking Mode is disabled
- Handle BLE permission states

Notes

- BLE scanning should be rate-limited to reduce battery drain.
 - Discovery should prioritize active and discoverable users.
 - UI should reflect scan status in real time.
-

6.3 NFC Module

Responsibility

Handles tap-based exchange of identifiers between devices.

Functions

- Detect supported NFC hardware
- Enable tap exchange session
- Send or receive minimal identity payload
- Trigger connection flow after tap

Notes

- NFC should exchange only a minimal token or user ID.
 - The module must fail gracefully on unsupported devices.
 - NFC should require app activity and explicit user intent.
-

6.4 Connection Manager

Responsibility

Coordinates connection creation, verification, and profile retrieval.

Functions

- Validate that both users are active
- Resolve exchanged user IDs
- Fetch profile details from local storage or backend
- Store connection history
- Manage undo/revoke actions

Notes

- This module acts as the bridge between connectivity and profile rendering.
 - It should unify BLE and NFC connection outcomes into one flow.
-

6.5 AI Engine

Responsibility

Generates networking insights from profile and connection data.

MVP Functions

- Create a readable profile summary from raw bio and interests
- Suggest reasons to connect
- Generate icebreaker prompts
- Rank nearby users by relevance

Future Functions

- API-backed inference
- Personalized matchmaking
- Event-specific relevance scores
- Relationship suggestions based on prior connections

Notes

- AI should augment the experience, not block core functionality.
 - The app must still work without AI.
-

7. Technology Stack

7.1 Frontend

- Android
- Kotlin
- Android Jetpack components recommended
- Material-style UI patterns for clarity and speed

7.2 Connectivity

- NFC APIs for tap communication
- BLE APIs for scanning and broadcasting

7.3 Storage

- SharedPreferences for lightweight settings
- Room database for structured local profile and connection data

7.4 AI Layer

- Rule-based logic in MVP
- API-based intelligence in later versions

7.5 Backend (Optional / Future)

- Firebase for authentication, sync, and real-time updates
 - Cloud storage for profile and connection history
 - Analytics pipeline for growth insights
-

8. Data Model

8.1 User Entity

Represents the TapConnect user profile.

Fields

- user_id: string
- name: string
- profile_image: string
- bio: string
- social_links: object or JSON map
- interests: list of strings
- role: string
- organization: string
- is_networking_active: boolean
- sharing_preferences: object
- created_at: timestamp
- updated_at: timestamp

Example Social Links Structure

- Instagram
 - LinkedIn
 - WhatsApp
 - GitHub
 - Website
-

8.2 Connection Entity

Represents a successful networking event between two users.

Fields

- connection_id: string
- user_id_1: string
- user_id_2: string
- connection_type: NFC | BLE
- timestamp: datetime
- notes: optional string
- status: active | revoked

Notes

- Connection records can later support reminders, analytics, and follow-up features.
-

8.3 Discovery Session Entity

Represents a temporary nearby networking session.

Fields

- session_id: string
 - user_id: string
 - discovery_mode: NFC | BLE
 - active: boolean
 - started_at: timestamp
 - ended_at: timestamp
-

9. Data Storage Strategy

9.1 Local Storage

Local storage is used in the MVP to keep the app lightweight and offline-friendly.

Stored Locally

- User profile
- Sharing preferences
- Connection history
- AI-generated summaries
- App settings

Recommended Approach

- SharedPreferences for flags and preferences

- Room database for structured entities like users and connections
-

9.2 Future Remote Storage

When scaling is needed, backend support can be introduced for:

- User profile sync
 - Cross-device access
 - Connection backup
 - Event-based rooms
 - Real-time discovery data
-

10. Data Flow

10.1 NFC Flow

1. User opens the app.
2. Networking Mode is enabled.
3. NFC tap occurs.
4. Devices exchange minimal user identifiers.
5. Connection Manager validates the session.
6. Profile data is loaded.
7. Connection screen and shared profile are displayed.
8. Connection history is saved locally.

NFC Design Notes

- Fastest connection path.
 - Best for direct person-to-person exchange.
-

10.2 BLE Flow

1. User opens the app.
2. BLE scanning begins when Networking Mode is ON.
3. Nearby discoverable users are detected.
4. Users are shown in a nearby list.
5. User selects a profile.
6. Connection Manager resolves identity and fetches profile.
7. Shared profile is displayed.
8. Connection is logged.

BLE Design Notes

- Works as fallback when NFC is unavailable.
 - Suitable for crowded environments.
-

11. API Design (Future)

These endpoints are not required for the MVP but define the expected backend shape.

11.1 Fetch Profile

GET /user/{id}

Purpose

Retrieve public profile data for a given user ID.

Response

- user_id
 - name
 - bio
 - photo
 - interests
 - social_links
 - visibility metadata
-

11.2 Store Connection

POST /connect

Purpose

Store a successful connection between two users.

Payload

- user_id_1
 - user_id_2
 - connection_type
 - timestamp
-

11.3 AI Suggestions

POST /ai/suggestions

Purpose

Generate AI-based networking recommendations.

Payload

- user profile
- nearby users list
- event context if available

Response

- summary
 - connection reasons
 - icebreakers
 - relevance ranking
-

12. Security Considerations

Core Security Rules

- User consent is required via Networking Mode.
- No background sharing without explicit action.
- Only minimal identifiers should be exchanged during discovery.
- Sensitive profile fields should be hidden unless explicitly shared.
- Local storage should be protected using secure Android storage practices.

Recommended Protections

- Encrypt stored sensitive values where possible.
 - Use session tokens instead of raw data when feasible.
 - Avoid exposing social links until connection is confirmed.
 - Validate discovery sessions before showing full profiles.
-

13. Privacy Model

TapConnect is designed around privacy-first networking.

Privacy Rules

- Users are invisible when Networking Mode is OFF.

- Discovery is temporary and opt-in.
- Sharing should be field-based, not all-or-nothing.
- Users can revoke visibility at any time.
- Users should be able to control what is shared before connection.

Privacy Objective

Ensure users feel safe turning on discovery in public event spaces.

14. Scalability Strategy

MVP Scaling

For the first release, all essential data remains local.

Future Scaling Path

- Introduce backend profile sync.
- Use Firebase for real-time discovery and connection persistence.
- Split AI features into server-side services.
- Add event rooms and shared context models.

Architectural Benefit

Because the design is modular, moving from local storage to cloud sync should not require rewriting the entire app.

15. Error Handling

The app must handle failures gracefully.

Error Scenarios

- NFC unsupported device
- BLE permission denied
- Bluetooth disabled
- No nearby users found
- Tap exchange failed
- Connection timeout
- Profile fetch failure
- Storage write failure

Handling Strategy

- Show clear user-facing messages.

- Offer retry actions where appropriate.
 - Fall back from NFC to BLE when possible.
 - Prevent app crashes by isolating connection failures.
 - Log non-sensitive technical details for debugging.
-

16. Performance Considerations

Target Performance Goals

- Nearby detection should feel near real-time.
- Core connection actions should complete in under 2 seconds where possible.
- UI transitions should remain smooth and lightweight.

Performance Strategies

- Limit BLE scan frequency.
 - Cache local profile data.
 - Use efficient list rendering.
 - Avoid heavy AI processing on the UI thread.
 - Keep network calls minimal in the MVP.
-

17. Technical Risks

Risk 1: BLE Instability

BLE behavior may differ across Android devices.

Risk 2: NFC Compatibility

Not all Android devices support NFC reliably.

Risk 3: Permission Complexity

Android permissions can affect onboarding and scanning reliability.

Risk 4: UI Lag

Overly complex screens or scanning operations may slow responsiveness.

Risk 5: Discovery Reliability

Crowded environments may reduce scan accuracy or increase noise.

18. Mitigation Strategies

- Add fallback logic between NFC and BLE.
 - Test on multiple Android models and OS versions.
 - Keep permission prompts short and clear.
 - Use retry logic for failed discovery or exchange attempts.
 - Reduce background work during active networking.
 - Keep the UI minimal and event-focused.
-

19. Implementation Phases

Phase 1: UI + Profile System

Deliverables

- Profile creation and editing
- Settings and sharing preferences
- Local data storage
- Basic UI structure

Phase 2: BLE Discovery

Deliverables

- BLE scanning
- Nearby user list
- Networking Mode toggle
- Discovery status indicators

Phase 3: NFC Integration

Deliverables

- NFC tap support
- Minimal identifier exchange
- Tap confirmation flow
- Connection animation

Phase 4: AI Features

Deliverables

- Smart summaries
- Icebreaker suggestions
- Connection reasoning

- Priority ranking

Phase 5: Optimization & Testing

Deliverables

- Device compatibility testing
 - Permission flow improvements
 - Performance tuning
 - UX refinement
 - Crash and failure handling
-

20. Deployment Strategy

Development Build

- Use Android Studio builds for internal development.
- Test using debug APKs on real devices.

Internal Testing

- Test across different device models.
- Verify NFC, BLE, and permission behavior.
- Simulate crowded event conditions.

Release Strategy

- Start with APK distribution for early testing.
 - Move to Play Store release after stability and polish.
-

21. Monitoring and Logging

MVP Monitoring

- Use Logcat for debugging during development.

Future Monitoring

- Firebase Analytics for event tracking
- Crash reporting for stability monitoring
- Metrics for discovery success, connection rates, and retention

Useful Metrics

- NFC success rate

- BLE discovery success rate
 - Permission acceptance rate
 - Connection completion time
 - App crash frequency
-

22. Testing Strategy

Functional Testing

- Profile creation
- Visibility toggle
- NFC exchange
- BLE discovery
- Connection history
- AI summary generation

Compatibility Testing

- Different Android versions
- NFC-enabled and NFC-disabled devices
- BLE behavior across device vendors

UX Testing

- Time to connect
- Ease of onboarding
- Visibility clarity
- Connection feedback quality

Security Testing

- Consent enforcement
 - Data visibility validation
 - Local storage safety
-

23. Dependencies

The project depends on:

- Android system NFC support
 - Android BLE APIs
 - Runtime permissions handling
 - Reliable local storage implementation
 - Future backend services if scaling is added
-

24. Out of Scope for MVP

The following are not required in the first release:

- Full chat system
 - Cross-platform support
 - Social feed
 - Event organizer dashboard
 - Advanced backend matchmaking
 - Multi-device cloud sync
 - Complex AI personalization engine
-

25. Conclusion

TapConnect's technical design focuses on fast proximity networking, local-first data handling, and privacy-aware discovery. The architecture supports immediate MVP delivery while leaving clear room for NFC enhancement, AI expansion, and backend scaling.

The result is a modular Android system built to make in-person networking faster, smarter, and more natural.